Лабораторна робота №4: Створення найпростішого HTTP-сервера на Node.js.

Мета роботи

- **Використати навички програмування на JavaScript та роботи з форматами JSON/XML**
- **Запрограмувати асинхронні виклики для читання файлу**
- **Навчитися обробляти запити та надсилати відповідь з допомогою вбудованого модуля HTTP**

Теоретичний матеріал

- Лекція 4 - Node.js модулі та структура проекту.Практика програмування з Node.js
- Лекція 5 - Практика програмування з Node.js
- Лекція 6 - Поняття про асинхронне програмування
- Лекція 5 - Практика програмування з Node.js
- Лекція 7 - Протокол HTTP. Простий сервер на Node.js

Підготовка до роботи

- Створіть **новий репозиторій з назвою backend-course-2025-4**
- Встановіть ім'я користувача у форматі **`Ваше Ім'я bc2025-4`** та вашу електронну пошту для коміту з допомогою команди git config
- Закомітте у ваш репозиторій json файл з даними відповідно до вашого варіанту
    - **Варіант 1** - **bank_managers.json** - Інформація про керівників банків, визначені в статті 42 Закону України "Про банки і банківську діяльність".
    - **Варіант 2** - **flights-1m.json** - Великий набір даних польотної інформації (1 мільйон записів), що включає затримки та скасування.
    - **Варіант 3** - **house-price.json** - Ціни на житло з різноманітними характеристиками та властивостями.
    - **Варіант 4** - **iris.json** - Класичний набір даних вимірювань квіток ірису.
    - **Варіант 5** - **mtcars.json** - Дані про споживання пального та метрики продуктивності різних моделей авто.
    - **Варіант 6** - **titanic.json** - Інформація про пасажирів "Титаніка", включно зі статусом виживання.

    - **Варіант 7** - **weather.json** - Дані про історичні показники погоди з різних метеостанцій.
- **Створіть файл package.json** у кореневій теці вашого репозиторію. Створіть головний файл програми. Закомітте обидва файли.
- **Встановіть пакет Commander.js** для роботи з командним рядком **локально** для вашого проекту Node.js.
    - **Створіть файл .gitignore** та переконайтеся, що тека з встановленими сторонніми модулями додана як виключення для Git.
    - Використайте модуль у програмі, щоб читати аргументи командного рядка.

Частина 1 - спільні параметри командного рядка

- Ваша програма (**незалежно від варіанту**) має приймати наступні аргументи:
    - **`-i, --input`** - (обов'язковий параметр) шлях до файлу, який даємо для читання
    - **`-h, --host`** - (обов'язковий параметр) адреса сервера
    - **`-p, --port`** - (обов'язковий параметр) порт сервера
- Програма має виводити помилку, якщо файл для читання не знайдено. Текст помилки: "Cannot find input file".
- Програма має виводити помилку, якщо не задано обов'язковий параметр.
- З допомогою модуля http запустіть веб сервер. Значення параметрів **`--host`**, **`--port`** мають бути передані у метод **`http.Server.listen()`**
- Для того, щоб сервер автоматично перезапускався кожного разу, коли ви змінюєте програму, запустіть сервер з допомогою пакета **`nodemon`**, цей пакет **встановіть як залежність лише для розробки**.
- Виконайте вимоги цієї частини та закомітте результат.

Частина 2 - читання файлу та обробка запиту

- Використовуйте **асинхронні** виклики r**eadFile та writeFile** для читання та запису файлів (модуль fs Node.js) у цій частині.
- Програма має виконувати завдання **незалежно від того, які значення мають поля прочитаного з файлу JSON**. Тобто якщо значення полів будуть інші, програма все одно має коректно виконувати свою задачу.
- Створіть програму яка запускає веб-сервер і для кожного запиту:
    - читає JSON з файлу
    - аналізує його вміст відповідно до завдання
    - формує XML і надсилає його у відповідь на запит до вашого серверу
- Для формування XML використайте пакет **`fast-xml-parser`**. Встановіть його, додайте до залежностей package.json. **Дослідіть документацію і у звіті окресліть основні особливості роботи з пакетом**.
- Виконайте завдання відповідно до варіанту нижче та закомітте

Варіант 1: bank_managers.json

Параметри URL query для фільтрації (аналог параметрів `-m, –mfo` та `-n, –normal`):

- **`?mfo=true`**: Відображати код МФО банку перед назвою (поле `MFO`).
- **`?normal=true`**: Відображати лише працюючі банки з кодом 1 “Нормальний” (фільтрація за полем `COD_STATE`).

Приклад виводу інформації у форматі XML (якщо задано обидва параметри, вихідні поля: `MFO`, `NAME`, `COD_STATE`):

```
<banks>
  <bank>
    <mfo_code>305880</mfo_code>
    <name>АТ "КБ "ЗЕМЕЛЬНИЙ КАПІТАЛ"</name>
    <state_code>1</state_code>
  </bank>
  <bank>
    <mfo_code>380731</mfo_code>
    <name>АТ "Дойче Банк ДБУ"</name>
    <state_code>1</state_code>
  </bank>
</banks>
```

Варіант 2: flights-1m.json

Параметри URL query для фільтрації (аналог параметрів `-d, –date` та `-a, –airtime`):

- **`?date=true`**: Відображати дату (поле `FL_DATE`) у результаті.
- **`?airtime_min=X`**: Відображати лише записи літаків, час у повітрі яких (`AIR_TIME`) був довший за заданий **X**.

Приклад виводу інформації у форматі XML (якщо `airtime_min` використаний, та `date=true`, вихідні поля: `FL_DATE`, `AIR_TIME`, `DISTANCE`):

```
<flights>
  <flight>
    <date>2006-01-01</date>
    <air_time>350</air_time>
    <distance>2475</distance>
  </flight>
  <flight>
    <date>2006-01-02</date>
    <air_time>343</air_time>
```

```
    <distance>2475</distance>
  </flight>
</flights>
```

Варіант 3: house-price.json

Параметри URL query для фільтрації (аналог параметрів `-f, –furnished` та `-p, –price`):

- **`?furnished=true`**: Відображати лише будинки, які мають статус “furnished” (фільтрація за полем `furnishingstatus`).
- **`?max_price=X`**: Відображати лише будинки з ціною (`price`) меншою за зазначену **X**.

Приклад виводу інформації у форматі XML (вихідні поля: `price`, `area`, `furnishingstatus`):

```
<houses>
  <house>
    <price>13300000</price>
    <area>7420</area>
    <furnishingstatus>furnished</furnishingstatus>
  </house>
  <house>
    <price>12250000</price>
    <area>8960</area>
    <furnishingstatus>furnished</furnishingstatus>
  </house>
</houses>
```

Варіант 4: iris.json

Параметри URL query для фільтрації (аналог параметрів `-v, –variety` та `-l, –length`):

- **`?variety=true`**: Відображати вид квітки (поле `variety`) у результаті.
- **`?min_petal_length=X`**: Відображати лише записи з довжиною пелюстки (`petal.length`) більшою за задану **X**.

Приклад виводу інформації у форматі XML (вихідні поля: `petal.length`, `petal.width`, `variety`):

```
<irises>
  <flower>
    <petal_length>1.4</petal_length>
    <petal_width>0.2</petal_width>
    <variety>Setosa</variety>
  </flower>
  <flower>
    <petal_length>6.7</petal_length>
    <petal_width>2.2</petal_width>
    <variety>Virginica</variety>
  </flower>
</irises>
```

Варіант 5: mtcars.json

Параметри URL query для фільтрації (аналог параметрів `-c, –cylinders` та `-m, –mpg`):

- **`?cylinders=true`**: Відображати кількість циліндрів (поле `cyl`) у результаті.
- **`?max_mpg=X`**: Відображати лише записи машин з палиною економністю (`mpg`) нижче за зазначену **X**.

Приклад виводу інформації у форматі XML (вихідні поля: `model`, `cyl`, `mpg`):

```
<cars>
  <car>
    <model>Mazda RX4</model>
    <cyl>6</cyl>
    <mpg>21</mpg>
  </car>
  <car>
    <model>Merc 450SL</model>
    <cyl>8</cyl>
    <mpg>17.3</mpg>
  </car>
</cars>
```

Варіант 6: titanic.json

Параметри URL query для фільтрації (аналог параметрів `-s, –survived` та `-a, –age`):

- **`?survived=true`**: Відображати лише пасажирів, які вижили (фільтрація за полем `Survived`).
- **`?age=true`**: Відображати вік пасажирів (поле `Age`) у результаті.

Приклад виводу інформації у форматі XML (вихідні поля: `Name`, `Age`, `Ticket`):

```
<passengers>
  <passenger>
    <name>Braund, Mr. Owen Harris</name>
    <age>22</age>
    <ticket>A/5 21171</ticket>
  </passenger>
  <passenger>
    <name>Palsson, Master. Gosta Leonard</name>
    <age>2</age>
    <ticket>349909</ticket>
  </passenger>
</passengers>
```

Варіант 7: weather.json

Параметри URL query для фільтрації (аналог параметрів `-h, –humidity` та `-r, –rainfall`):

- **`?humidity=true`**: Відображати вологість вдень (поле `Humidity3pm`) у результаті.
- **`?min_rainfall=X`**: Відображати лише записи з фактичною кількістю опадів (`Rainfall`) більше за зазначену **X**.

Приклад виводу інформації у форматі XML (вихідні поля: `Rainfall`, `Pressure3pm`, `Humidity3pm`):

```
<weather_data>
  <record>
    <rainfall>0</rainfall>
    <pressure3pm>1015</pressure3pm>
    <humidity>29</humidity>
  </record>
  <record>
    <rainfall>3.6</rainfall>
    <pressure3pm>1008.4</pressure3pm>
    <humidity>36</humidity>
  </record>
</weather_data>
```

Результат:

- як мінімум 5 комітів виконаних з командного рядка
- файл package.json якій містить використані у проекті пакети
- файл .gitignore відповідно до вимог
- файл main.js якій містить код програми, яка відповідає всім вимогам відповідно до варіанту
- файл якій містить дані

Максимальна оцінка: 4 бали